

Apache Airflow DAG Pipeline using Docker

1. Introduction

Apache Airflow is an open-source platform for programmatically authoring, scheduling, and monitoring workflows. Pipelines are needed to orchestrate and automate data workflows instead of running manual scripts. Docker is used to run Airflow in a containerized environment, avoiding dependency and OS issues.

2. Airflow Features

Some of the most important features of Apache Airflow are:

1. **Directed Acyclic Graphs (DAGs):**
 - Workflows are defined as code in Python.
 - Each DAG represents a pipeline with tasks and dependencies.
2. **Task Orchestration & Scheduling:**
 - The scheduler ensures that tasks are executed in the right order.
 - Supports both time-based scheduling (cron) and event-driven triggers.
3. **Rich Web UI:**
 - Airflow provides an interactive UI to view, manage, and monitor DAGs.
 - Views available: **DAG Graph View, Tree View, Gantt Chart, and Logs.**
4. **Scalability:**
 - Can run on a single machine or scale to multiple workers using **Celery** or **Kubernetes executors**.
5. **Extensible Operators:**
 - Airflow supports many built-in operators like **BashOperator, PythonOperator, SQL Operators, etc.**
 - Custom operators can also be created.
6. **Connections and Hooks:**
 - Reusable connections to databases, APIs, and cloud services.
 - Hooks provide standard interfaces to external systems like MySQL, Postgres, AWS, Azure, and GCP.
7. **Retry and Alert Mechanism:**
 - Tasks can be retried automatically on failure.
 - Email/SMS/Slack alerts can be configured for failures.
8. **Integration with Ecosystem:**
 - Works with big data tools (Spark, Hadoop), cloud storage, and data warehouses.

These features make Airflow a **powerful tool for building, automating, and monitoring complex workflows** in real-world data engineering projects.

3. Setup & Installation

Steps followed to set up Airflow using Docker:

1. Installed Docker Desktop (AMD64 for my system).
2. Downloaded project materials including docker-compose.yaml.
3. Created a .env file with Airflow configurations such as username and password.
4. Started containers using:

docker compose up -d
5. Verified that Postgres, Redis, Airflow Webserver, and Scheduler containers were healthy using:

docker ps

4. DAG Creation

Created a new DAG file tuto.py inside the dags/ folder. Below is a simplified code snippet:

```
from airflow import DAG
from airflow.operators.bash_operator import BashOperator
from datetime import datetime, timedelta

default_args = {
    "owner": "airflow",
    "depends_on_past": False,
    "start_date": datetime(2024, 1, 1),
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

dag = DAG("tutorial", default_args=default_args,
          schedule_interval=timedelta(days=1))

t1 = BashOperator(task_id="print_date", bash_command="date", dag=dag)
t2 = BashOperator(task_id="sleep_task", bash_command="sleep 5",
                  dag=dag)

t1 >> t2
```

This DAG has two tasks:

1. Print the current system date.

2. Sleep for 5 seconds.

Airflow UI automatically detected this DAG.

5. Building Pipeline Steps and Views

Steps to Build and Run the Pipeline:

1. **Write the DAG** in Python inside the dags/ directory.
2. **Define tasks** using operators (e.g., BashOperator, PythonOperator).
3. **Set dependencies** between tasks (t1 >> t2).
4. **Start Airflow containers** (docker compose up -d).
5. Airflow Scheduler automatically **detects the DAG**.
6. Open **Airflow Web UI** at <http://localhost:8080>.
7. **Enable the DAG** using the toggle button.
8. **Trigger the DAG manually** or wait for its schedule.
9. **Monitor the DAG execution** in different views.

DAG Views in Airflow:

- **Graph View:** Visual representation of task flow.
- **Tree View:** Status of all past DAG runs.
- **Gantt Chart:** Shows execution timeline of tasks.
- **Logs:** Task logs for debugging and auditing.

These visualization features make it easy to understand dependencies and troubleshoot errors.

6. Creating a Connection

In the Airflow Web UI:

1. Navigated to Admin → Connections.
2. Created a Postgres connection with details like host, user, password, and database name.
3. This connection can be reused in DAGs for database-related tasks (ETL pipelines, inserts, queries).

This demonstrates Airflow's ability to integrate seamlessly with external databases.

7. Conclusion

In this mini project, Apache Airflow was successfully set up using Docker. A DAG pipeline was created with multiple tasks and executed through the Airflow UI. This demonstrated how Airflow helps in managing workflows and pipelines. Future enhancements can include integrating with databases, ETL pipelines, and cloud storage.